

CoffeeLib 2.0

Auditoria Arquitetural

Biblioteca Java de gerenciamento de configuracoes multiplataforma para Minecraft, cobrindo Paper (1.8.8+), Forge 1.20.1 e NeoForge 1.21.1 a partir de um nucleo comum sem qualquer acoplamento ao Minecraft.

Escopo	Do scaffold inicial ate o sistema de injecao de dependencias
Modulos	5 (api, core, platform-paper, platform-forge-1.20.1, platform-neoforge-1.21.1)
Commits auditados	17
Arquivos Java	38 em main, 7 em test
Testes automatizados	24, todos passando
Decisoes registradas	18 decisoes arquiteturais
Grupo Maven	net.loyalnetwork
Build	Gradle 8.14.5, daemon em JDK 21, piso de toolchain Java 17

Este documento registra **o que foi construido, como foi construido e por que cada decisao foi tomada** — incluindo as alternativas descartadas e o custo assumido em cada escolha. Decisoes revertidas e funcionalidades abandonadas estao registradas com o mesmo peso das aceitas: elas explicam a forma final do projeto.

1. Sumario executivo

A CoffeeLib entrega uma API unica de configuracao que funciona identicamente em tres plataformas de Minecraft com sistemas de configuracao nativos completamente diferentes. O desenvolvedor escreve uma classe anotada e nunca precisa saber qual backend esta em uso.

O que foi construido

- **coffeelib-api** — anotacoes, interfaces e excecoes. Zero dependencias externas.
- **coffeelib-core** — reflection, validacao, ciclo de vida e o container de DI. Depende apenas de api; nunca do Minecraft.
- **coffeelib-platform-paper** — backend YAML proprio que preserva comentarios, cobrindo 1.8.8+ com um unico jar.
- **coffeelib-platform-forge-1.20.1** e **coffeelib-platform-neoforge-1.21.1** — integracao real com ForgeConfigSpec / ModConfigSpec, registrando as configs no sistema nativo de cada loader.
- **Sistema de DI** provider/receiver baseado em anotacoes, inteiramente em core, sem codigo especifico de plataforma.

Como foi construido: verificacao empirica sobre memoria

Um metodo foi aplicado de forma consistente e merece destaque porque evitou multiplos erros: **nenhuma assinatura de API de plataforma foi escrita de memoria ou a partir de documentacao**. Antes de escrever os adapters de Forge e NeoForge, as classes reais foram extraidas dos jars resolvidos pelo Gradle e inspecionadas com javap; onde o comportamento nao era visivel na assinatura, os fontes oficiais foram lidos diretamente.

Verificacao	Fonte consultada	O que teria dado errado sem isso
Assinaturas de ModConfigSpec, ModContainer, ModConfigEvent	javap nos jars neoforge 21.1.248 e loader 4.0.43	Chamadas para metodos inexistentes ou overloads errados
Assinaturas de ForgeConfigSpec, ModLoadingContext	javap nos jars forge 47.4.22 e fmlcore	Mesmo risco; a API do Forge diverge do NeoForge em pontos sutis
Propagacao de evento por hierarquia no Forge	Fonte de ListenerList.java (eventbus 6.2.33)	Registrar dois listeners desnecessarios, ou um listener que nunca dispara
Campos obrigatorios de mods.toml	Fonte de ModInfo.java (fmlloader 47.4.22)	Jar rejeitado pelo loader — de fato aconteceu na primeira tentativa
Contrato de ILoadedConfig (sealed)	javap com flag de PermittedSubclasses	Teste impossivel de compilar sem entender a restricao

Resultado direto do metodo

A inspecao dos jars revelou um bug real no proprio codigo da CoffeeLib antes de chegar em producao: o uso de `List.of(...)` na lista de valores aceitos de `@OneOf`. A rotina de correcao do Forge e do NeoForge chama `.contains(null)` sobre essa colecao ao validar uma chave ausente do arquivo, e `List.of(...).contains(null)` lanca `NullPointerException` — diferente de `Arrays.asList`. Sem executar contra a API real, o erro so apareceria para o usuario final.

2. Arquitetura de modulos

A regra estrutural inegociavel do projeto: **o nucleo jamais depende do Minecraft**. Toda integracao especifica vive nos modulos de plataforma, e nenhum deles conhece os outros.

2.1 Grafo de dependencias

```
coffeelib-api (sem dependencias)
  ^
coffeelib-core api(project(":coffeelib-api"))
  ^
  +-- coffeelib-platform-paper + spigot-api 1.8.8 (compileOnly)
  +-- coffeelib-platform-forge-1.20.1 + forge 47.4.22 (ForgeGradle 6)
  +-- coffeelib-platform-neoforge-1.21.1 + neoforge 21.1.248 (ModDevGradle)
```

Cada modulo de plataforma declara `api(project(":coffeelib-core"))`, de modo que o desenvolvedor final declara **uma unica dependencia** — a da sua plataforma — e recebe `api` e `core` transitivamente, sem precisar saber que existem.

2.2 Organizacao de pacotes

Os pacotes foram reorganizados por dominio quando a biblioteca deixou de ser exclusivamente de configuracao. A estrutura deixa espaco explicito para novos dominios como irmaos de `config`, sem reabrir a hierarquia:

Pacote	Conteudo
<code>api.exception</code>	<code>CoffeeLibException</code> — raiz unchecked de toda falha da biblioteca
<code>api.config</code>	<code>ConfigManager</code> (fachada do dev), <code>ConfigBackend</code> (seam core/plataforma)
<code>api.config.annotation</code>	<code>@ConfigFile</code> , <code>@Comment</code> , <code>@Range</code> , <code>@OneOf</code> , <code>@Key</code> , <code>@Ignore</code>
<code>api.config.exception</code>	<code>ConfigException</code> e subtipos de load / save / validacao
<code>api.di</code>	<code>ServiceContainer</code>
<code>api.di.annotation</code>	<code>@Provide</code> , <code>@Receive</code>
<code>api.di.exception</code>	<code>ServiceException</code> , <code>AmbiguousProviderException</code> , <code>MissingProviderException</code>
<code>core.config</code>	<code>DefaultConfigManager</code> , <code>ConfigReflector</code> , <code>FieldCodec</code> , <code>ConfigValidator</code> , <code>Suggestions</code>
<code>core.di</code>	<code>DefaultServiceContainer</code> , <code>ProviderEntry</code>

2.3 O seam entre core e plataforma

A peca que sustenta a portabilidade e a interface `ConfigBackend`. Ela move uma estrutura chave-valor aninhada para dentro e para fora de um arquivo, sem qualquer nocao da classe Java anotada que o core esta populando:

```
String fileExtension();
Map<String, Object> read(Path file);
void write(Path file, Map<String, Object> values, Map<String, String> comments);
```

Secoas aninhadas sao Map aninhados; chaves de comentario usam o caminho pontilhado do valor que anotam. O core nunca sabe se o destino e YAML, TOML ou outra coisa.

3. Decisoes arquiteturais

Cada decisao abaixo foi tomada explicitamente, com alternativa considerada e custo assumido. Estao em ordem cronologica aproximada.

AD-01 API baseada em instancia, nao em campos estaticos		Aceita
CONTEXTO	A proposta inicial expunha valores como campos publicos estaticos (<code>ServerConfig.HOST</code>), acessiveis sem obter qualquer instancia.	
DECISAO	Migrar para instancias: o desenvolvedor obtem um <code>ConfigManager</code> e guarda a referencia da config carregada.	
POR QUE	Campos estaticos criam dois problemas concretos. Primeiro, o reload reescreve estado global por reflection, sem garantia de visibilidade entre threads. Segundo, impedem multiplas instancias da mesma config (por mundo, por contexto). Migrar de estatico para instancia depois seria uma quebra total de API — a decisao precisava ser tomada no dia 1.	
CONSEQUENCIA	Perde-se o acesso direto sem instancia. O desenvolvedor carrega uma vez na inicializacao e mantem a referencia.	

AD-02 Isolamento por composicao, nao por classloader		Aceita
CONTEXTO	Requisito: cada plugin/mod deve ter ambiente isolado dos demais, mesmo com um unico artefato em execucao.	
DECISAO	O core nao mantem nenhum estado estatico. Cada <code>ConfigManager</code> e dono do proprio registro como campo de instancia.	
POR QUE	Isolamento via classloader e uma ilusao especifica de plataforma. No Paper cada plugin tem seu <code>PluginClassLoader</code> , mas no NeoForge todos os mods compartilham o mesmo classloader — a duplicacao de classes por mod nunca existiu ali. Uma arquitetura apoiada nisso quebraria no Forge/NeoForge no primeiro dia.	
CONSEQUENCIA	Dois donos na mesma JVM carregando a mesma classe de config nunca colidem, e o mecanismo e identico nas tres plataformas, sem codigo especifico para resolver isolamento.	

AD-03 Um artefato publicado por plataforma		Aceita
CONTEXTO	Definir como a biblioteca seria consumida: artefato unico ou multiplos.	
DECISAO	Cada modulo de plataforma publica seu proprio artefato, expondo core e api como dependencias transitivas via <code>api(...)</code> .	
POR QUE	O desenvolvedor deve declarar exatamente uma dependencia — a da plataforma dele — e nunca precisar conhecer a existencia dos modulos internos.	
CONSEQUENCIA	Cada modulo continua compilavel isoladamente, e o build agregado produz todos os jars numa pasta unica.	

AD-04 Sem modulo compartilhado entre Forge e NeoForge		Aceita
CONTEXTO	Forge e NeoForge tem APIs de configuracao quase identicas. Foi proposto um modulo <code>forgeLike-common</code> concentrando a logica e deixando apenas adapters finos por versao.	
DECISAO	Rejeitado. Cada par (loader, versao) tem implementacao propria e completamente independente, mesmo com codigo semelhante.	
POR QUE	O NeoForge se distancia progressivamente do Forge; uma abstracao compartilhada quebraria sozinha em algum ponto futuro, e o custo de desfaze-la seria maior que o da duplicacao. A granularidade real de divergencia nao e nem "loader", e sim (loader, versao) — Forge 1.16.5 e 1.20.1 ja divergiam entre si.	
CONSEQUENCIA	Custo assumido: uma feature nova no core precisa ser propagada manualmente a cada adapter, sem nada estrutural forçando consistencia. Testes de contrato sao a mitigacao recomendada quando a matriz crescer.	

AD-05 Descartar suporte a Forge 1.16.5		Aceita (escopo reduzido)
CONTEXTO	Era um alvo desejado de suporte legado.	
DECISAO	Removido do escopo, junto com o modulo correspondente.	
POR QUE	Tres fatos empilhados. NeoForge nao existe para 1.16.5 (nasceu na 1.20.1), entao seria Forge classico. O Forge 1.16.5 exige ForgeGradle 5, que suporta Gradle no maximo 7.x. E o Gradle nao permite versoes distintas por subprojeto — e uma unica versao de wrapper para todo o build. Manter 1.16.5 exigiria um build standalone separado, com publicacao manual do core como pre-requisito.	
CONSEQUENCIA	Decisao do proprietario do projeto: <i>"era um capricho, mas esta ficando caro demais"</i> . O limite inferior de suporte forge-like passa a ser 1.20.1.	

AD-06 Gradle 8.14.5 com daemon fixado em JDK 21		Aceita
CONTEXTO	O projeto comecou em Gradle 9.3.0 com JDK 25, e o modulo Forge 1.20.1 falhava ao configurar.	
DECISAO	Rebaixar o wrapper para 8.14.5 e apontar <code>org.gradle.java.home</code> para a JDK 21.	
POR QUE	Duas incompatibilidades independentes. ForgeGradle 6 nao suporta Gradle 9.x. E o compilador Kotlin embutido no Gradle 8.x nao consegue interpretar a string de versao do JDK 25, falhando ao compilar qualquer <code>script.gradle.kts</code> .	
CONSEQUENCIA	O daemon roda em JDK 21, mas cada modulo mantem seu proprio toolchain de compilacao — sao configuracoes independentes.	

AD-07 Piso de toolchain Java 17 para api, core e paper		Aceita
CONTEXTO	Com o daemon em JDK 21, esses modulos passaram a compilar para bytecode 21, quebrando a dependencia do modulo Forge 1.20.1 (toolchain fixado em 17).	
DECISAO	Fixar Java 17 como piso no build raiz, permitindo que modulos com exigencia maior sobrescrevam localmente.	
POR QUE	O Gradle recusa que um modulo com toolchain menor dependa de artefato com bytecode maior. 17 e o menor denominador comum entre as plataformas configuradas — o Minecraft 1.20.1 distribui Java 17 ao usuario final.	
CONSEQUENCIA	NeoForge 1.21.1 sobrescreve para Java 21 no proprio script, como Mojang exige na 1.21.	

AD-08 Comentarios preservados no Paper via serializador proprio		Aceita
CONTEXTO	O <code>YamlConfiguration.save()</code> do Bukkit descarta comentarios ao gravar — historicamente a maior limitacao do sistema nativo.	
DECISAO	Escrever um leitor/escritor YAML proprio no modulo Paper, em vez de delegar ao Bukkit.	
POR QUE	Preservar comentarios foi declarado como a motivacao original para criar a biblioteca . Aceitar a limitacao do Bukkit anularia boa parte do proposito do projeto.	
CONSEQUENCIA	O formato so precisa concordar consigo mesmo, entao permanece deliberadamente estreito: pares chave-valor indentados, secoes aninhadas em profundidade ilimitada e comentarios em linha propria. Nao suporta listas — limitacao consciente, alinhada ao que o core produz hoje.	

AD-09 Validacao falha explicitamente, sem correcao silenciosa		Aceita
CONTEXTO	Definir o comportamento de <code>@Range</code> ao encontrar valor fora dos limites: ajustar para o limite mais proximo ou recusar.	
DECISAO	Lancar <code>ConfigValidationException</code> . Para <code>@OneOf</code> em texto, a execucao carrega sugestoes ordenadas por distancia de edicao.	
POR QUE	Ajuste silencioso mascara erro de operador — um erro de digitacao numa porta viraria um servidor rodando em configuracao nao pretendida, sem aviso. Falhar alto e mais seguro.	
CONSEQUENCIA	A execucao expoe <code>fieldName</code> , <code>invalidValue</code> e <code>suggestions</code> como dados estruturados, permitindo que o consumidor construa a propria mensagem sem parsear texto.	

AD-10 Aninhamento recursivo com deteccao de ciclo		Aceita
CONTEXTO	A implementacao inicial suportava apenas campos de primeiro nivel.	
DECISAO	Reflection recursiva de profundidade ilimitada, com pilha de tipos detectando auto-referencia em tempo de scan.	
POR QUE	Requisito explicito de suportar quantos objetos aninhados forem necessarios. A deteccao de ciclo evita recursao infinita em tempo de carregamento, com mensagem nomeando o caminho.	
CONSEQUENCIA	Instancias aninhadas sao reutilizadas no reload em vez de recriadas, preservando referencias que o codigo do desenvolvedor ja tenha guardado.	

AD-11 Integração nativa com o sistema de config de cada loader		Aceita
CONTEXTO	O backend poderia apenas gravar TOML válido, ou registrar-se de fato no sistema de configuração do loader.	
DECISÃO	Registro completo via <code>ModContainer#registerConfig</code> (NeoForge) e <code>ModLoadingContext#registerConfig</code> (Forge), traduzindo as anotações para <code>ModConfigSpec</code> / <code>ForgeConfigSpec</code> .	
POR QUE	Requisito explícito: outros mods e os comandos <code>/forge config</code> e <code>/neoforge config</code> precisam enxergar as configs. Gravar um arquivo TOML isolado não entrega isso.	
CONSEQUÊNCIA	Divergência de comportamento assumida: a plataforma passa a ser dona da validação. Nos módulos forge-like, um valor fora de <code>@Range</code> é corrigido silenciosamente para o padrão com aviso em log, em vez de lançar exceção como no Paper. Consequência inerente de delegar a posse, não um defeito.	

AD-12 Código de integração vive na plataforma, nunca no core		Aceita
CONTEXTO	A tradução de metadados de campo para <code>ModConfigSpec</code> poderia residir no core e para evitar duplicação.	
DECISÃO	Cada módulo forge-like faz seu próprio scan por reflection das anotações públicas da API.	
POR QUE	Decisão explícita do proprietário: o core é agnóstico. <code>ModConfigSpec</code> é um alvo estruturalmente distinto — um builder com estado, push/pop, produzindo valores vinculados — e não o mapa plano que o core gera.	
CONSEQUÊNCIA	Reflection semelhante existe em três lugares. Custo aceito em troca de manter o núcleo livre de qualquer conceito de plataforma.	

AD-13 Sistema de DI deliberadamente mínimo		Aceita
CONTEXTO	Pedido de um sistema provide/receive baseado em anotações.	
DECISÃO	<code>@Provide</code> em getter sem argumentos, <code>@Receive</code> em campo, com nome opcional para desambiguar. Correspondência por tipo exato.	
POR QUE	Quatro regras definidas pelo proprietário eliminaram a maior parte da complexidade: singleton (getter roda no máximo uma vez), quebra explícita quando a ordem está errada, falha no carregamento em 100% dos casos sem resolução opcional, e nomes para desambiguar. Sem grafo de dependência entre providers, não há necessidade de ordenação topológica.	
CONSEQUÊNCIA	Não há widening por supertipo — um provider que retorna <code>ArrayList</code> não satisfaz um campo declarado <code>List</code> . Ambiguidade sem nome produz exceção listando os candidatos e sugerindo a anotação nomeada.	

AD-14 Versionamento e migracao de config: descartado		Rejeitada
CONTEXTO	Proposta de migradores tipados encadeados, lendo a versao gravada no arquivo e aplicando transformacoes v1 para v2 para v3.	
DECISAO	Nao implementar.	
POR QUE	O desenho so encaixa naturalmente onde a CoffeeLib e dona do formato do arquivo — ou seja, apenas no Paper. Nos modulos forge-like a posse do arquivo foi deliberadamente entregue ao loader (AD-11), sem acesso ao conteudo bruto antes do processamento, e o sistema de correcao deles nao tem conceito de renomear chave. Migracao real exigiria ler o TOML manualmente antes do registro, migrar e regravar.	
CONSEQUENCIA	Decisao do proprietario: manter paridade entre plataformas vale mais que uma funcionalidade que so funcionaria bem em uma delas.	

AD-15 Jars distribuiveis como mod/plugin standalone		Aceita
CONTEXTO	Os jars nao continham descritor de mod nem classe principal, e eram rejeitados ao serem colocados nas pastas mods/ ou plugins/.	
DECISAO	Adicionar descritores e uma classe principal trivial, sem logica, em cada modulo de plataforma.	
POR QUE	Confirmado empiricamente contra clientes reais de NeoForge 1.21.1, Forge 1.20.1 e Paper. Estabelece que a CoffeeLib funciona como biblioteca standalone, e nao apenas embutida dentro de outro artefato.	
CONSEQUENCIA	No Paper, o plugin.yml deliberadamente omite api-version — declara-lo exigiria fixar uma versao moderna do Bukkit e anularia o suporte a 1.8.x a partir de um unico jar.	

AD-16 Pasta dist/ na raiz com nomes de jar distintos		Aceita
CONTEXTO	Nao havia local unico e previsivel para coletar todos os artefatos apos o build.	
DECISAO	./gradlew build passa a agregar todos os jars em ./dist.	
POR QUE	Requisito de poder compilar tudo individualmente e tambem tudo junto, com saida unificada.	
CONSEQUENCIA	Expos uma colisao real: os dois modulos forge-like definiam o nome do artefato como o mod_id, que e coffeeLib em todas as plataformas por design. Os jars tinham nome identico e o Gradle abortou a copia. Cada modulo passou a ter nome explicito e distinto.	

AD-17 Excecao raiz comum a toda a biblioteca		Aceita
CONTEXTO	Com a chegada do dominio de DI, ConfigException deixou de ser uma raiz adequada.	
DECISAO	Introduzir CoffeeLibException em api.exception, com ConfigException e ServiceException estendendo-a.	
POR QUE	Da a dominios futuros uma base unchecked compartilhada sem acopla-los ao tipo de execucao de configuracao.	
CONSEQUENCIA	Todas as excecoes permanecem unchecked: sao problemas de operador ou de desenvolvedor, nao condicoes de recuperacao em tempo de execucao.	

AD-18 Um unico modulo Paper para 1.8.8 ate a versao atual		Aceita
CONTEXTO	Definir se o Paper precisaria de um modulo por versao, como o Forge.	
DECISAO	Um modulo unico, compilado contra <code>org.spigotmc:spigot-api:1.8.8</code> .	
POR QUE	A superficie do Bukkit usada — <code>JavaPlugin</code> e o acesso ao diretorio de dados — e estavel desde a 1.8, sem quebra binaria. O repositório do PaperMC não publica <code>paper-api</code> anterior a 1.9.4, e a API do Spigot é a base que o Paper 1.8 de fato implementava.	
CONSEQUENCIA	Ao contrario do Forge, não há custo de manutenção por versao no Paper — um jar cobre toda a faixa suportada.	

4. Problemas encontrados e resolvidos

Registro dos defeitos e incompatibilidades reais enfrentados durante a construçao. Nenhum deles era previsível a partir de documentação — todos apareceram na execução.

4.1 Ambiente e toolchain

Sintoma	Causa raiz	Resolucao
Compilacao falhava com <code>ExceptionInInitializerError</code> no processador do Lombok	Lombok 1.18.34 não suporta JDK 25; falha ao ler internos do javac	Atualizado para Lombok 1.18.42
Worker de teste do Gradle não encontrava as classes de teste, apesar de compiladas em disco	Caminho do projeto continha acentos (Área de Trabalho), quebrando o mecanismo de argfile de classpath no Windows	Projeto movido para <code>C:\project</code> . Um workaround de encoding tentado antes foi removido por não ser a causa
Erro de sintaxe Kotlin no script do Forge	O MDK oficial do Forge é escrito em Groovy; foi colado num arquivo <code>.gradle.kts</code>	Módulos forge-like passaram a usar <code>build.gradle</code> (Groovy). Mistura de DSL por subprojeto e suportada
Plugin <code>net.minecraftforge.gradle</code> não encontrado	Faltava <code>pluginManagement</code> apontando para o repositório do Forge	Repositórios de plugin adicionados ao settings
Could not get unknown property 'mod_version'	O template do MDK espera um <code>gradle.properties</code> por módulo que não acompanhava o script	Arquivo criado com os valores reais do projeto

4.2 Código e integração

Sintoma	Causa raiz	Resolucao
<code>NullPointerException</code> na validação do NeoForge, revelado por teste	<code>@OneOf</code> usava <code>List.of(...)</code> ; a correção do loader chama <code>.contains(null)</code> ao validar chave ausente	Trocado para <code>Arrays.asList</code> , que tolera null. Correção aplicada preventivamente também no Forge
Skipping jar. File ... is not a valid mod file	Jar sem descritor de mod e sem classe <code>@Mod</code> construível	Descritores templatzados e classe principal trivial adicionados
Missing required field mandatory in dependency	Forge 1.20.1 exige o campo booleano <code>legado mandatory=true</code> , não o <code>type="required"</code> do NeoForge	Confirmado lendo <code>ModInfo.java</code> dos fontes do fmlloader e corrigido
Missing metadata in pack mod:coffeelib	Tudo jar de mod também é tratado como resource pack; faltava <code>pack.mcmeta</code>	Arquivo adicionado nos dois módulos forge-like
Falha ao copiar jars para dist/	Forge e NeoForge produziam jars com nome idêntico (ambos derivados de <code>mod_id</code>)	Nome de artefato explícito e distinto por módulo
Classpath do NeoForge/Forge ausente na compilação de testes	ModDevGradle e ForgeGradle só conectam o classpath ao source set principal	Source set de teste passou a estender o classpath de main

5. Cobertura de testes

24 testes automatizados, todos passando. A estratégia priorizou executar contra as APIs reais das plataformas em vez de duplicar seu comportamento em dublês.

Modulo	Testes	O que cobrem
core.config	8	Defaults na primeira execucao, leitura de valores em disco, violacao de @Range, sugestao de @OneOf por distancia de edicao, reload atualizando a instancia existente, aninhamento profundo, reuso de instancia aninhada e rejeicao de ciclo
core.di	7	Resolucao sem nome com provider unico, resolucao nomeada, ambiguidade, ausencia de provider, nome inexistente, falha explicita por ordem incorreta e garantia de execucao unica do getter
platform-paper	4	Round-trip do formato YAML com preservacao de comentarios e secoes aninhadas; integracao completa via JavaPlugin mockado, com reflection e I/O de arquivo reais
platform-forge	3	Construcao real de ForgeConfigSpec com contexto estatico interceptado, registro de listener e ligacao do event bus
platform-neoforge	2	Construcao real de ModConfigSpec percorrendo dois niveis de aninhamento, e registro dos listeners de Loading e Reloading

Limite conhecido dos testes de plataforma

Os testes de Forge e NeoForge validam a construcao real da especificacao de configuracao — essa parte nao exige jogo em execucao. O que nao e coberto e o ciclo completo de carregamento a partir do disco: assim que algo precisa ser corrigido ou gravado, o codigo do loader alcanca FMLPaths e o ConfigTracker reais, que so existem num ambiente de mod carregado. Essa fronteira esta documentada no proprio teste, em vez de ser contornada com dublês que dariam falsa confianca. A validacao dessa camada foi feita manualmente, executando os jars em clientes reais.

6. Debito tecnico e pontos de atencao

Itens conhecidos, deliberadamente nao resolvidos, com o motivo registrado.

Item	Impacto	Observacao
Paridade entre adapters nao e forçada	Medio	Consequencia direta de AD-04. Uma feature nova no core precisa ser propagada manualmente aos tres adapters. Testes de contrato no core sao a mitigacao recomendada quando a matriz de versoes crescer
Divergencia de validacao entre plataformas	Medio	No Paper, valor invalido lanca excecao; nos forge-like, e corrigido para o padrao com aviso em log. Inerente a AD-11. Esta documentado no Javadoc de cada manager, mas o desenvolvedor precisa conhece-lo
APIs deprecadas no Forge 1.20.1	Baixo	ModLoadingContext.get() e FMLJavaModLoadingContext.get() estao marcados para remocao, ainda que sejam o padrao documentado atual. Sinaliza migracao para injecao via construtor, como o NeoForge ja faz
Formato YAML do Paper nao suporta listas	Baixo	Alinhado ao que o core produz hoje. Suporte a listas exigiria trabalho coordenado nos dois lados

Item	Impacto	Observacao
Configuration cache do Gradle desligado	Baixo	ForgeGradle 6 tem historico de incompatibilidade; nao foi verificado. As demais otimizacoes (paralelismo, build cache) estao ativas
Migracao de config ausente	A definir	Rejeitada em AD-14 por nao encaixar nos modulos forge-like. Renomear um campo hoje faz o valor antigo ser perdido e o padrao ser aplicado
Modulo api sem testes proprios	Nenhum	Contem apenas anotacoes, interfaces e excecoes sem comportamento. Exercitado indiretamente por todos os demais modulos

7. Historico de commits

17 commits, em ordem cronologica.

Hash	Descricao
dbadd6e	Scaffold multi-modulo e implementacao de core + plataforma Paper
63678cc	Wrapper do Gradle para 8.14.5 e daemon fixado em JDK 21
0e57f91	Java 17 como piso de toolchain para api, core e paper
09d3d2b	Propriedades do MDK para o modulo Forge 1.20.1
43211b4	Remocao de boilerplate nao utilizado do script do Forge (datagen, gametest)
6b6e105	Sincronizacao do nivel de linguagem do IntelliJ com o piso de toolchain
ccc161d	Resolver foojay para provisionamento automatico de toolchain
a7b1c17	Build paralelo, cache de build e limite de heap do daemon
4d800db	Script de build do modulo NeoForge 1.21.1
c318e4f	Ignorar artefatos de runtime gerados pelas tasks de Forge/NeoForge
c09e4bd	Integracao de config do NeoForge via registro de ModConfigSpec
6534953	Integracao de config do Forge via registro de ForgeConfigSpec
2c14495	Pasta dist/ na raiz com nomes de jar distintos
54e2ffd	Jars de Forge e NeoForge carregaveis como mods standalone
2d2687d	Reorganizacao de pacotes por dominio e execucao raiz comum
2c630c8	Jar do Paper carregavel como plugin standalone
4750640	Sistema de injecao de dependencias provider/receiver

8. Conclusao

A biblioteca cumpre o objetivo central definido no inicio: **o desenvolvedor escreve uma classe de configuracao anotada e nunca precisa saber qual backend esta em uso**. As tres plataformas suportadas compartilham a mesma API publica, com o nucleo permanecendo completamente livre de qualquer dependencia do Minecraft, de YAML ou de TOML.

Duas caracteristicas do processo merecem registro por terem afetado diretamente a qualidade do resultado. A primeira e a verificacao empirica sistematica contra as APIs reais das plataformas, que capturou um defeito genuino no proprio codigo da biblioteca e evitou multiplas suposicoes incorretas sobre assinaturas e comportamento. A segunda e a disposicao de **reduzir escopo quando o custo se mostrou desproporcional** — o suporte a Forge 1.16.5 e o sistema de migracao de configuracoes foram descartados por razoes tecnicas concretas, e nao por dificuldade de implementacao. Ambas as decisoes preservaram a coerencia do projeto.

O ponto de atencao mais relevante para a manutencao a longo prazo e a ausencia de um mecanismo estrutural que garanta paridade entre os adapters de plataforma. E consequencia consciente da decisao de nao compartilhar codigo entre Forge e NeoForge, e o momento natural para introduzir testes de contrato no nucleo sera quando a matriz de versoes suportadas crescer.